

■ Design Notification System

System Design Interview | Asynchronous Systems

algomaster.io | Deep Dive Notes

■ Key Requirements

Functional	Send push/email/SMS/in-app notifications; Support templates; Scheduling; Delivery tracking; User preferences & opt-out
Non-Functional	High throughput (1M+ notifications/min); Reliable delivery (at-least-once); No duplicate delivery; Low latency for urgent notifis
Scale	Transactional notifis: <1s latency; Marketing notifis: batch delivery OK; Handle provider outages gracefully

■ High-Level Architecture

- API layer accepts notification requests → validates → publishes to Kafka topic by channel (email/push/SMS/in-app)
- Channel Workers consume from Kafka → enrich with user data → apply template → call third-party provider
- Providers: APNs (iOS push), FCM (Android push), SendGrid/SES (email), Twilio/SNS (SMS)
- Delivery Tracker: webhook from providers updates delivery status in DB; retry engine handles failures

■ Reliability & Deduplication

- **Idempotency Key:** Caller provides unique notification_id; worker checks DB/Redis before sending; prevents duplicates on retry
- **At-Least-Once:** Kafka consumer commits offset only after successful send to provider (or max retries); guaranteed delivery
- **Retry with Backoff:** Exponential backoff with jitter for provider failures; dead letter queue (DLQ) for exhausted retries; alert ops team
- **Circuit Breaker:** If FCM error rate >10% in 60s → open circuit; route to backup provider or queue for later; auto-close after recovery

■ Push Notification Deep Dive

- iOS: APNs requires device token registered by app; token stored in user-device table; token expires/rotates
- Android: FCM registration token; can use topic-based messaging for group/broadcast instead of individual sends
- Token management: device tokens change → store per-device, per-user; stale token detection via provider feedback
- Priority: FCM supports high/normal priority; APNs supports time-sensitive/active-interruption; use appropriately to avoid iOS throttling

■ User Preferences & Rate Limiting

- Preference store: per-user, per-channel, per-category settings (e.g., marketing emails: off; order updates: on)

- Quiet hours: user-defined do-not-disturb windows; timezone-aware scheduling; urgent alerts bypass quiet hours
- Notification rate limiting: max N notifications per user per day per channel; prevents notification fatigue
- Aggregation: batch similar notifications (e.g., '5 people liked your post' vs 5 separate push notifications)

■ Queue Strategy

Priority Queue	Separate Topics
✓ Single queue with priority	✓ Separate Kafka topics per priority
✓ Urgent messages first	✓ Independent scaling
✓ Simpler topology	✓ No cross-contamination
✓ Head-of-line blocking risk	✓ More consumer groups

■ Monitoring & Analytics

- Delivery metrics: sent, delivered, failed, bounced, clicked (email open tracking via pixel)
- Alert on: delivery rate drop, provider error spike, queue depth increase, DLQ growth
- Funnel analytics: notification sent → delivered → opened → converted; A/B test notification content